

LAB ASSIGNMENT – 20.2

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Question:

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

- Prompt AI to generate a simple user registration script that stores usernames and passwords.
- Check whether passwords are stored as plain text.
- Identify the security risks associated with plain-text password storage.
- Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
- Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Prompt:

Generate a simple Python program for user registration and login that stores usernames and passwords. The program should initially store passwords in plain text.

Code:

```
import hashlib
```

```
def hash_password(password):
```

```
    return hashlib.sha256(password.encode()).hexdigest()
```

```
def register_user(users):
```

```
    username = input("Enter username: ")
```

```
    if username in users:
```

```
        print("Username already exists")
```

```
    return
```

```
    password = input("Enter password: ")
```

```
    users[username] = hash_password(password)
```

```
    print("Registration successful")
```

```
def login_user(users):
```

```
    username = input("Enter username: ")
```

```
    password = input("Enter password: ")
```

```
    if username in users and users[username] == hash_password(password):
```

```
        print("Login successful")
```

```
    else:
```

```
        print("Invalid credentials")
```

```
users = {}
```

```
while True:
```

```
choice = input("r-register l-login q-quit: ")
```

```
if choice == 'r':
```

```
    register_user(users)
```

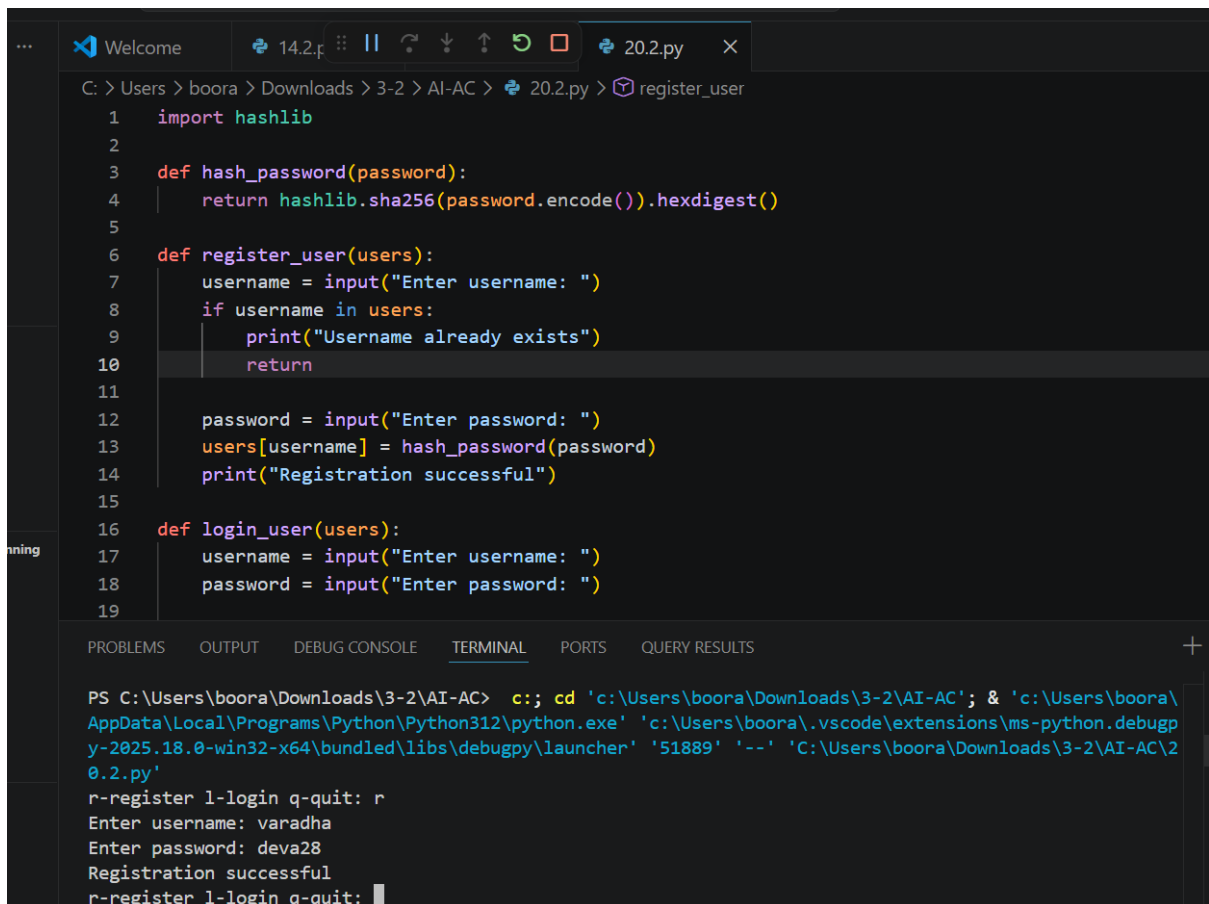
```
elif choice == 'l':
```

```
    login_user(users)
```

```
elif choice == 'q':
```

```
    break
```

Output:



The screenshot shows a VS Code editor window with a Python file named 20.2.py. The code defines a hash_password function using hashlib, a register_user function that checks for existing usernames and hashes passwords, and a login_user function. The terminal output shows the command prompt running the script, followed by the user input 'r' to register, then 'varadha' as the username and 'deva28' as the password, resulting in a 'Registration successful' message. The prompt is then ready for the next input.

```
C: > Users > boora > Downloads > 3-2 > AI-AC > 20.2.py > register_user
1  import hashlib
2
3  def hash_password(password):
4      return hashlib.sha256(password.encode()).hexdigest()
5
6  def register_user(users):
7      username = input("Enter username: ")
8      if username in users:
9          print("Username already exists")
10         return
11
12         password = input("Enter password: ")
13         users[username] = hash_password(password)
14         print("Registration successful")
15
16  def login_user(users):
17      username = input("Enter username: ")
18      password = input("Enter password: ")
19
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\boora\Downloads\3-2\AI-AC> c:; cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51889' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\20.2.py'
r-register l-login q-quit: r
Enter username: varadha
Enter password: deva28
Registration successful
r-register l-login q-quit: 
```

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and

improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

Prompt:

Generate a Python script that connects to an external API using a hardcoded API key and fetches some data.

Code:

```
import os
```

```
import requests
```

```
def fetch_data():
```

```
    api_key = os.getenv("API_KEY")
```

```
    if not api_key:
```

```
        print("API key not found!")
```

```
    return
```

```
url = "https://jsonplaceholder.typicode.com/posts"
```

```
headers = {"Authorization": f"Bearer {api_key}"}
```

```
try:
```

```
    response = requests.get(url, headers=headers)
```

```
    response.raise_for_status()
```

```
    data = response.json()
```

```
    print("Data fetched successfully:")
```

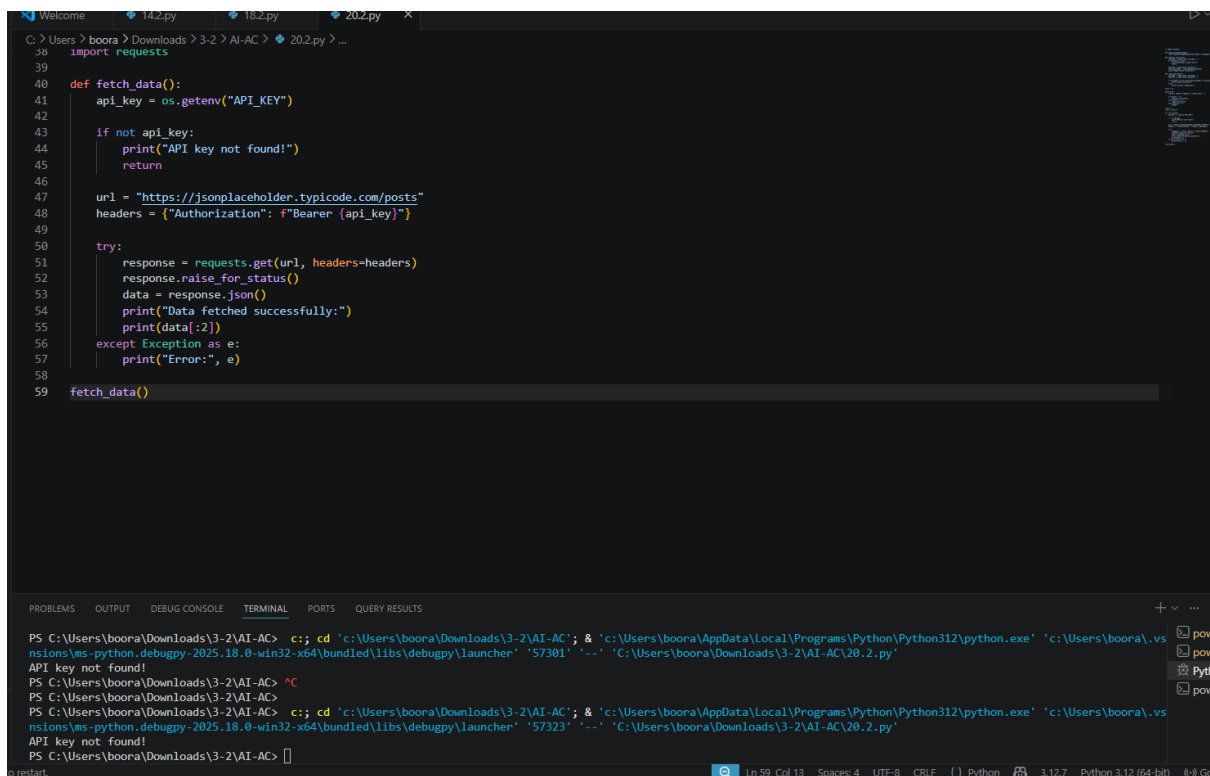
```
    print(data[:2])
```

```
except Exception as e:
```

```
    print("Error:", e)
```

```
fetch_data()
```

Output:



The screenshot shows a Visual Studio Code editor with a Python script named `20.2.py` open. The script defines a function `fetch_data()` that attempts to fetch data from a JSON placeholder API. The script includes error handling for missing API keys and network exceptions. The terminal output shows that the API key was not found, resulting in an "API key not found!" message.

```
def fetch_data():
    api_key = os.getenv("API_KEY")
    if not api_key:
        print("API key not found!")
        return
    url = "https://jsonplaceholder.typicode.com/posts"
    headers = {"Authorization": f"Bearer {api_key}"}
    try:
        response = requests.get(url, headers=headers)
        response.raise_for_status()
        data = response.json()
        print("Data fetched successfully:")
        print(data[:2])
    except Exception as e:
        print("Error:", e)

fetch_data()
```

Terminal Output:

```
PS C:\Users\boora\Downloads\3-2\AI-AC> cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\boora\vs
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57301' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\20.2.py'
API key not found!
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC>
PS C:\Users\boora\Downloads\3-2\AI-AC> cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\boora\vs
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57323' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\20.2.py'
API key not found!
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).
- Refactor the program using stronger encryption techniques available in Python libraries.
- Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Prompt:

Generate a simple Python program that encrypts and decrypts a message using a basic method like Base64 encoding.

Code:

```
import base64

msg = input("Enter message: ")

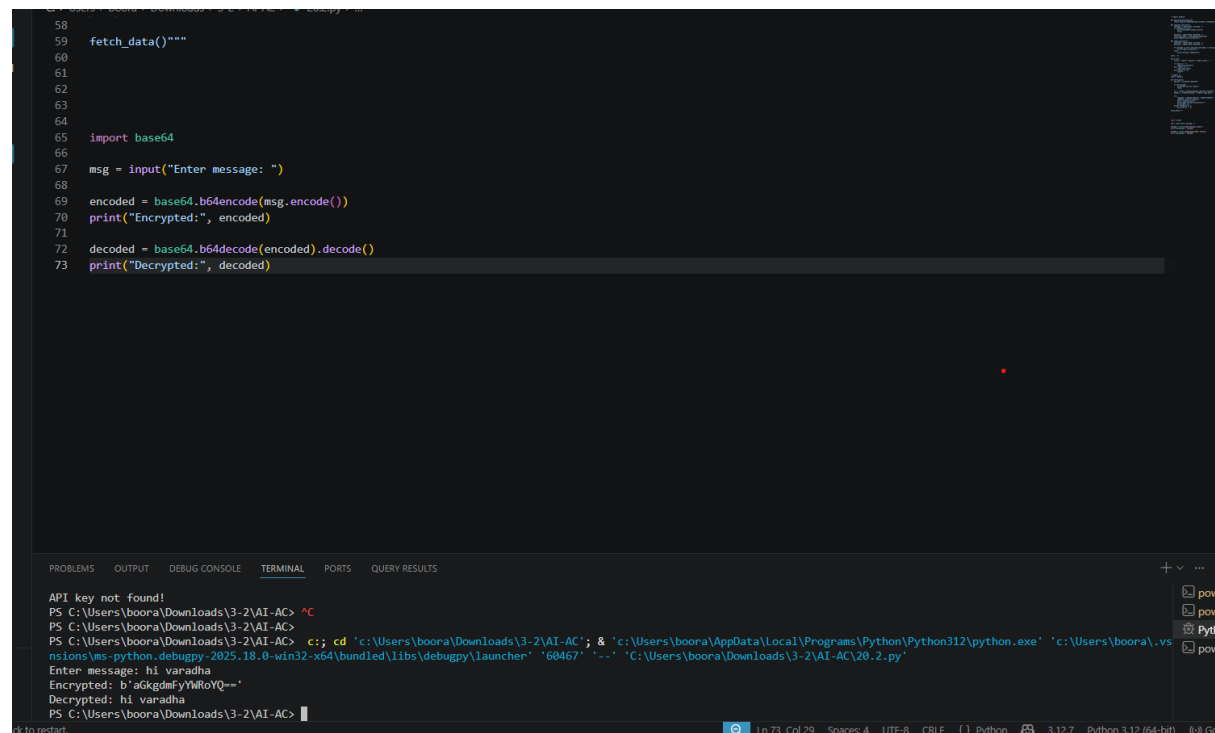
encoded = base64.b64encode(msg.encode())

print("Encrypted:", encoded)
```

```
decoded = base64.b64decode(encoded).decode()
```

```
print("Decrypted:", decoded)
```

Output:



```
58
59 fetch_data()"""
60
61
62
63
64
65 import base64
66
67 msg = input("Enter message: ")
68
69 encoded = base64.b64encode(msg.encode())
70 print("Encrypted:", encoded)
71
72 decoded = base64.b64decode(encoded).decode()
73 print("Decrypted:", decoded)
```

API key not found!

```
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC>
PS C:\Users\boora\Downloads\3-2\AI-AC> cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\boora\vs
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60467' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\20.2.py'
Enter message: hi varadha
Encrypted: b'aGkgdmFyYWRoYQ==
Decrypted: hi varadha
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Task 4 – Real-Time Application: Security Review of AI-Generated

Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured

version.

- Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

Prompt:

Generate a simple Flask-based login API that accepts username and password and returns a success or failure response.

Code:

```
from flask import Flask, request, jsonify
```

```
import hashlib
```

```
app = Flask(__name__)
```

```
# Secure password storage (hashed)
```

```
users = {
```

```
    "admin": hashlib.sha256("1234".encode()).hexdigest()
```

```
}
```

```
@app.route('/')
```

```
def home():
```

```
    return "Server is running"
```

```
@app.route('/login', methods=['POST'])
```

```
def login():
```



```
data = request.get_json()
```

```
# Input validation
```

```
if not data or 'username' not in data or 'password' not in data:
```

```
    return jsonify({"error": "Invalid input"}), 400
```

```
username = data['username']
```

```
password = hashlib.sha256(data['password'].encode()).hexdigest()
```

```
# Authentication
```

```
if username in users and users[username] == password:
```

```
    return jsonify({"message": "Login successful"}), 200
```

```
else:
```

```
    return jsonify({"error": "Invalid credentials"}), 401
```

```
if __name__ == "__main__":
```

```
    app.run(debug=False)
```

Output:

```
79 import hashlib
80
81 app = Flask(__name__)
82
83 # Secure password storage (hashed)
84 users = {
85     "admin": hashlib.sha256("1234".encode()).hexdigest()
86 }
87
88 @app.route('/')
89 def home():
90     return "Server is running"
91
92 @app.route('/login', methods=['POST'])
93 def login():
94     data = request.get_json()
95
96     # Input validation
97     if not data or 'username' not in data or 'password' not in data:
98         return jsonify({"error": "Invalid input"}), 400
99
100     username = data['username']
101     password = hashlib.sha256(data['password'].encode()).hexdigest()
102
103     # Authentication
104     if username in users and users[username] == password:
105         return jsonify({"message": "Login successful"}), 200
106     else:
107         return jsonify({"error": "Invalid credentials"}), 401
108
109 if __name__ == "__main__":
110     app.run(debug=False)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
* Debugger PIN: 159-361-059
127.0.0.1 - - [16/Apr/2026 23:36:52] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [16/Apr/2026 23:36:52] "GET /favicon.ico HTTP/1.1" 404 -
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC> c:; cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\boora\.vs
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '49797' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\20.2.py'
* Serving Flask app '20.2'
* Debug mode: off
```

Un 110 Col 25 Spaces 4 UTF-8 CRUF Python 3.12.7 Python 3.12 (64-bit)

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents

crashes and displays user-friendly error messages for invalid inputs.\

Prompt:

Generate a simple Python program that takes two numbers as input and performs division without handling any errors.

Code:

```
def divide():  
    try:  
        a = float(input("Enter first number: "))  
        b = float(input("Enter second number: "))  
  
        result = a / b  
        print("Result:", result)  
  
    except ValueError:  
        print("Invalid input! Please enter numeric values.")  
  
    except ZeroDivisionError:  
        print("Error! Division by zero is not allowed.")  
  
    except Exception as e:  
        print("Unexpected error:", e)  
  
divide()
```

Output:

```
110 | app.run(debug=False)"""
111 |
112 |
113 |
114 |
115 | def divide():
116 |     try:
117 |         a = float(input("Enter first number: "))
118 |         b = float(input("Enter second number: "))
119 |
120 |         result = a / b
121 |         print("Result:", result)
122 |
123 |     except ValueError:
124 |         print("Invalid input! Please enter numeric values.")
125 |
126 |     except ZeroDivisionError:
127 |         print("Error! Division by zero is not allowed.")
128 |
129 |     except Exception as e:
130 |         print("Unexpected error:", e)
131 |
132 | divide()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC>
PS C:\Users\boora\Downloads\3-2\AI-AC> c:: cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\bo
nsions\ms-python-debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '58045' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\20.2.py'
Enter first number: 1
Enter second number: 2
Result: 0.5
PS C:\Users\boora\Downloads\3-2\AI-AC> 
```

Ln 132, Col 9 Spaces: 4 UTF-8 CRLF Python 3.12.7 Python 3

Justifications:

Task 1 – Password Security

Plain-text passwords are insecure as they can be easily exposed during data breaches. Using hashing (SHA-256) ensures passwords are stored securely and cannot be reversed.

Task 2 – API Key Management

Hardcoding API keys exposes sensitive credentials and risks misuse if code is shared. Using environment variables protects secrets and keeps them outside the source code.

Task 3 – Encryption Improvement

Base64 encoding is not secure since it is easily reversible and provides no protection. Strong encryption like Fernet ensures data confidentiality using a secret key.

Task 4 – Web Service Security

The original code had vulnerabilities like plain-text passwords, no validation, and debug exposure.

Applying hashing, validation, and secure responses improves security and reliability.

Task 5 – Exception Handling

Lack of error handling can cause program crashes due to invalid input or runtime errors. Using try-except blocks ensures stability and provides user-friendly error messages.